Section A:

Question 1:
The Order Run: I started with a straight dictionary attack using the main wordlist (`xato-net-10-million-passwords.txt`), followed by rule-based attacks (applying mutation rules like Jumbo and Best64), then targeted mask and hybrid attacks (testing patterns like words combined with numbers and symbols), and finally single/lookup modes (like CrackStation and username mutations).

Justification & Rationale: This order follows the principle of computational efficiency (going from cheapest/fastest to most expensive).

1. Dictionary attacks: Dictionary attacks test exact matches instantly, catching weak, unmodified passwords right away.
2. Rule Based Attacks: Rule-based attacks take those same dictionary words and apply common human modifications (capitalization, appending numbers like 123, special characters like `!`), which cracked the bulk of the middle-tier passwords.
3. Mask and brute-force/incremental modes are much slower because they expand the search space exponentially, so I saved them for stubborn hashes once obvious dictionary terms are ruled out. Information from earlier runs (such as seeing passwords like `Imhe23` or `Dewalt123`) directly informed my next choices by revealing that users favored capitalized words paired with numeric suffixes.

Question 2:
Mode & Count: The dictionary and rule-based attacks (`--wordlist` combined with rule sets like Jumbo/Best64) cracked the vast majority of the set—approximately 36 out of the 45 hashes.

Implications on Password Selection: This proves that human beings do not choose truly random passwords. Instead, they rely on predictable habits like picking standard dictionary words, capitalizing the first letter, and appending common numeric strings or symbols (e.g., `minecraft`, `50spanks`, `Dewalt123`). This predictability makes dictionary and rule mutation the most effective attack vector against standard users.

Question 3:
Mode & Count: Pure brute-force / incremental modes (without a dictionary or structural hints) cracked the fewest hashes (0).

Required Structure to Resist: To resist mask-based and dictionary/rule attacks, a password must lack predictable patterns, meaning it cannot be based on a dictionary word, a proper name, a common substitution, or a standard numerical sequence (like a birth year or PIN). Instead, it must be either a long, multi-word passphrase or a random string containing a mix of upper/lowercase letters, numbers, and symbols of sufficient length.

Section B:

Question 4:

What it tells me: Because the plaintext password is known to be in the wordlist, a failure under a straight dictionary attack proves that the stored hash was transformed or modified prior to hashing. The raw plaintext word was not hashed directly as-is.

1.Rule-Based Attack (`-r` / rules): Test if the password has common prefixes, suffixes, or character substitutions applied to it (e.g., capitalizing or appending numbers).

2. Salt Check / Format Verification: Check if the hash is salted (e.g., `md5(password + salt)`).

3.Alternative Transformations: Check for advanced permutations like reversed strings, double-hashing, or case inversion.

Question 5:
Identification & Hashcat Modes:

- 32-character hex strings = MD5 (Hashcat mode `-m 0`)
- 64-character hex strings = SHA-256 (Hashcat mode `-m 1400`)
- Strings starting with $2b12$ = Bcrypt (Hashcat mode `-m 3200`)

First: MD5 (32-char hex) — Attack this first because it is computationally trivial for GPUs to process (billions of hashes per second), yielding maximum cracked credentials quickly within limited time.

Second: SHA-256 (64-char hex) — Attack second. It is slower than MD5 due to iterative design, but still fast enough to brute-force weak passwords on modern hardware.

Last: Bcrypt ($2b12$) — Attack last (or skip if time is severely constrained). Bcrypt is intentionally designed with an adaptive cost factor (12) to be computationally expensive and memory-hard, making mass GPU cracking economically unviable.

Relationship to Algorithm Design: Fast hashing algorithms (MD5/SHA-256) place the security burden on the user to pick long, complex passwords because attackers can test millions per second cheaply. Slow, salted adaptive hashing algorithms (Bcrypt) shift the burden to the defender by making hardware brute-forcing prohibitively expensive for attackers, even if a user chooses a weaker password.

Section C:

Exact Benchmark Outputs:
MD5 (hashcat -b -m 0): Speed.#02.....: 6503.0 MH/s (6,503,000,000 hashes/second)
Bcrypt (hashcat -b -m 3200): Speed.#02.....: 4241 H/s(4241 hashes/second)

Ratio = (MD5 speed)/(Bcrypt speed) = 6,503,000/4241 = 1,533,364.77245933

Physical Meaning: An attacker can compute approximately 1.53 million MD5 password hashes in the exact same time it takes to compute just one bcrypt hash. This huge

discrepancy means that MD5 permits massive, high-speed parallel GPU brute-forcing of weak passwords, whereas bcrypt's intentional slowness makes large-scale offline guessing computationally unfeasible.

Question 7:
1. Keyspace Size

Formula: 26^8

Calculation: 26 * 26 * 26 * 26 * 26 * 26 * 26 * 26 = 208,827,064,576

2. MD5 Time Calculation

Hash Rate: 6,503,000,000 h/s

Time = 208,827,064,576/6,503,000,000 = 32.11 seconds

3. Bcrypt Time Calculation

Hash Rate = 4241 h/s

Time = 208,827,064,576/4241 = 49,240,052,952.4 seconds = 1561.39 years

Section D:

Question 8:

Part 1:
Conclusion about the accounts: Without cracking anything, we can conclusively determine that these two user accounts share the exact same plaintext password.

What an attacker learns and implications: If an attacker cracks one user's password, they immediately know the plaintext password for the second account (credential reuse). For an organization storing an entire user table without salts, this means a single compromised hash compromises every account that happened to choose the same password, enabling widespread credential stuffing.

Part 2:

Why rainbow tables work against unsalted.txt vs. salted.txt: Rainbow tables work by precomputing massive lookup chains of plaintext-to-hash mappings for a fixed, static algorithm (like raw MD5). Because unsalted.txt uses no salt, a single precomputed rainbow table can reverse any hash in the file.

Why it fails against salted hashes: In salted.txt (md5(password + salt)), every user has a unique random salt appended, meaning the mathematical hash function changes for every single user row. To attack this with a rainbow table, an attacker would have to precompute a completely separate, massive rainbow table for every single unique salt. Because storage

and generation costs scale linearly with the number of unique salts, precomputing millions of custom rainbow tables for each salt is computationally and storage-wise infeasible.

Question 9:

Does the salt defeat rainbow table attacks here? Yes. Even though the attacker has the salt, rainbow tables require a single, static precomputed hash function. Because each user has a distinct salt, the attacker would still need a separate rainbow table for every unique salt value, rendering rainbow tables useless.

Does the salt slow down a local hashcat attacker who has both the hash and the salt? No. In an offline brute-force attack where the hash and salt are already known, Hashcat simply appends the local salt to each dictionary/mask candidate (md5(candidate + salt)). This adds a negligible amount of overhead, meaning the hash rate remains virtually identical to unsalted MD5 (millions of hashes per second).

What property of the hash algorithm limits how fast an attacker can work? The computational efficiency / low work factor of the underlying hash function (i.e., MD5 was intentionally engineered to be extremely fast to compute in hardware/software, lacking any iterative stretching or memory-hardness).

Question 10:

What part of the claim is correct? The developer is correct that adding a unique random salt defeats rainbow table attacks and protects against precomputation.

What critical vulnerability remains? The critical vulnerability that remains is that MD5 is a fast cryptographic hash function. Because it lacks computational cost factors, an attacker who obtains the database can still execute high-speed offline brute-force or dictionary attacks against individual accounts using GPUs at rates of millions of hashes per second.

Exhaustive Search Time Estimation (6-character all-lowercase password):

Keyspace: $26^6 = 308,915,776$ possible passwords.

MD5 Speed (from Section C): $6,503,000,000 H/s$.

$$MD5Time = \frac{308,915,776}{6,503,000,000} \approx 0.0475 seconds$$

Bcrypt Speed (from Section C): $4,241 H/s$.

$$BcryptTime = \frac{308,915,776}{4,241} \approx 72,840 seconds \approx 20.2 hours$$

Does the speed difference matter at this password length? Yes, absolutely. At 6 characters, MD5 can be fully brute-forced in under 0.05 seconds—meaning a salt provides *zero* practical

security if a user picks a short or weak password. In contrast, Bcrypt takes over 20 hours to brute-force the exact same 6-character keyspace, meaning Bcrypt's slow speed successfully protects weak passwords from rapid offline cracking.

Section E:

Question 12:

Library Selected: bcrypt .

Why it is better than hashlib.md5(): Modern secure password storage requires key-stretching algorithms designed to be computationally slow and memory-hard. Standard MD5 was engineered for fast checksumming, making it vulnerable to massive parallel brute-forcing on modern GPUs. Bcrypt incorporates a configurable work factor (rounds) that intentionally limits cracking speeds regardless of attacker hardware.

Quantifying the Difference (from Section C benchmarks):

MD5 benchmark speed: $6,503,000,000 H/s$

Bcrypt benchmark speed: $4,241 H/s$

Bcrypt is ~1,533,365 times slower than MD5 on identical hardware, making offline brute-forcing computationally prohibitive for attackers.

Question 13:

Result: The two generated output strings are not identical.

What this reveals: The library automatically generates a unique, cryptographically secure random salt for every function call and embeds that salt directly inside the resulting hash string.

Byte Encoding:

The function first converts both string inputs (password and stored_hash) into UTF-8 encoded byte arrays (password_bytes and hash_bytes) using .encode("utf-8") because bcrypt.checkpw() operates on raw bytes.

Parsing the Stored Hash (bcrypt.checkpw internal step):

Inside bcrypt.checkpw(password_bytes, hash_bytes), bcrypt parses the hash_bytes structure.

Re-Hashing & Computation:

Using the extracted salt and cost parameter from the stored hash, bcrypt hashes the candidate password_bytes.

Constant-Time Comparison & Return:

Bcrypt performs a constant-time comparison between the newly generated hash and hash_bytes to prevent timing attacks.

If they match, bcrypt.checkpw() returns True; if they differ or if an error/invalid format occurs, it returns False (handled by the try...except block)

Why Plaintext Storing is Unnecessary: Plaintext passwords never need to be saved in the database because bcrypt.checkpw reconstructs the candidate hash deterministically using the salt embedded inside the stored hash string. Comparing two hashes proves whether the candidate password was identical to the original.

Question 14:

Rainbow Table Attacks:

Explanation: Plain MD5 storage allows attackers to look up hashes in precomputed rainbow tables instantly. The fixed_storage.py implementation uses automatic, unique per-user salts, rendering precomputed tables useless because a separate table would be required for every unique salt.

High-Speed GPU Brute-Force / Mask Attacks:

Explanation: Plain MD5 allows attackers to attempt billions of password candidates per second. The fixed_storage.py implementation uses Bcrypt with an adaptive work factor (cost=12), slowing the attacker down to only thousands of hashes per second and making exhaustive brute-forcing infeasible.

Section F:

1. Minimum Length (2 pts)

Policy Requirement: Minimum length of 12 characters.

Cracking Evidence: Short passwords were cracked almost instantaneously during Phase 3. Specifically, passwords like so16 (4 chars), uv40 (4 chars), ryub (4 chars), 19871 (5 chars), 23262 (5 chars), and 05323 (5 chars) were all easily broken in under a second using basic incremental/digit masks.

2. Required Character Classes (2 pts)

Policy Requirement: Require at least 3 out of 4 character classes (uppercase, lowercase, numbers, special symbols), prohibiting single-class or simple dual-class passwords.

Cracking Evidence:

Single-class numeric passwords (100801, 635100, 990703) succumbed immediately to digit masks (?d?d?d?d?d?d).

Simple dual-class word+number combinations (Linux123, Dewalt123, Imhe23, zappa8945) fell quickly to basic rule-based dictionary transformations (--rules=Jumbo).

3. Dictionary Blacklist (2 pts)

Policy Requirement: Yes. Implement an automated blacklist checking against the Top 10 Million common password dictionary and block passwords containing common roots or brand names.

Cracking Evidence & Attack Tier Defeated: Over 80% of cracked hashes (minecraft, 50spanks, kittykat, poopie, mexican, judy) were direct dictionary entries. A dictionary blacklist completely neutralizes Phase 1 Straight Dictionary Attacks and significantly weakens Phase 2 Rule-Based Mutations.

4. Attack Resistance Mapping (2 pts)

Minimum Length (12+): Defeats Brute-Force / Mask Attacks because expanding keyspaces requires centuries to compute, even on fast hash algorithms.

Dictionary Blacklist: Defeats Dictionary Attacks by preventing users from choosing words listed in public dumps (xato-net-10-million-passwords.txt).

Character Diversity: Defeats Simple Mask Attacks (e.g., pure numeric or all-lowercase patterns) by forcing the attacker to search across the full ASCII character set (95^N).

5. Usability Cost (2 pts)

Inconvenience: Requiring a minimum length of 12 characters with mixed complexity makes it more difficult to remember passwords, leading users to write passwords down or reuse them across platforms if not paired with a password manager.

Justification: Yes, the security benefit fully justifies the cost. Short passwords can be cracked in under 0.05 seconds under weak storage (MD5). Pairing a 12+ character policy with an enterprise password manager mitigates user inconvenience while ensuring computational resistance against offline cracking.